

Introduction au langage Assembleur

BUT Informatique à Arles

Ricardo Uribe Lobello

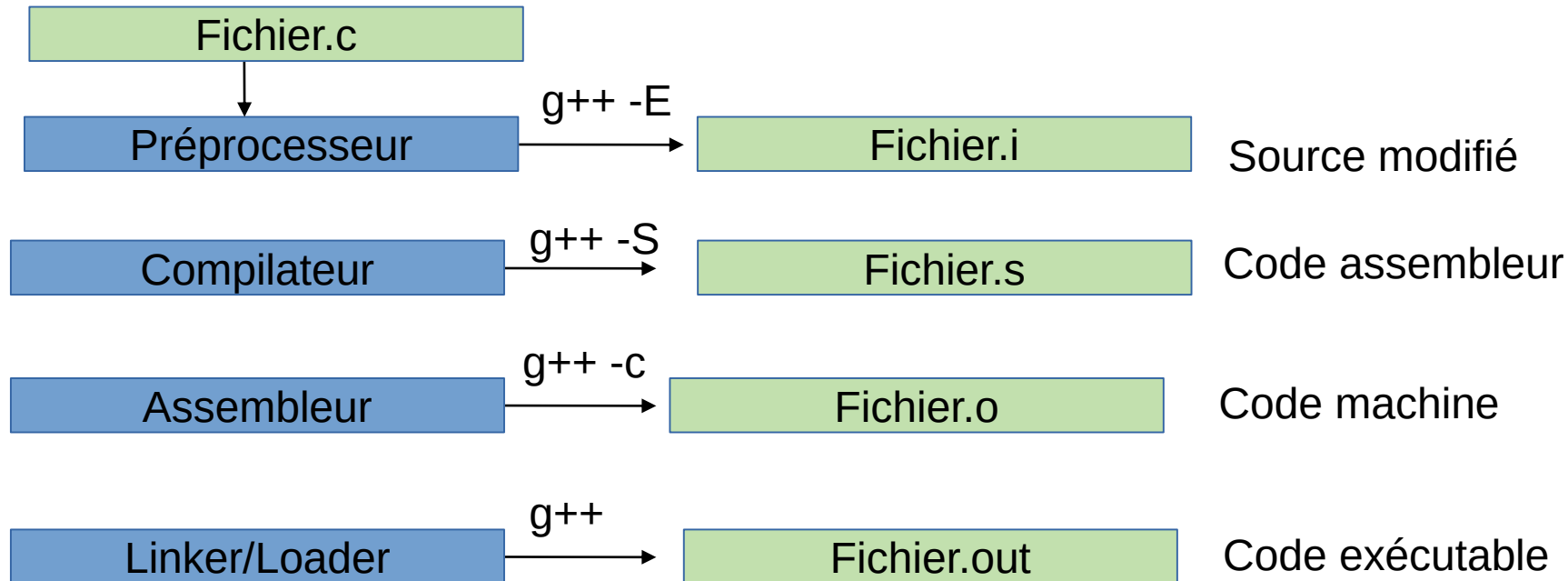
Programmer en assembleur

Pourquoi programmer en assembleur ?

- Le code écrit en assembleur peut être plus compacte et plus rapide que le code qui a été généré par un compilateur.
- Avec l'assembleur, il est possible d'accéder à des fonctions matérielles du système de manière plus directe. Cela n'est pas toujours possible avec un langage de haut niveau.
- En programmant en assembler, le programmeur peut acquérir une compréhension plus approfondie de la manière comment les ordinateurs fonctionnent.
- L'assembleur permet de mieux comprendre le fonctionnement des langages de haut niveau et surtout des compilateurs qui génèrent le code machine.
- Programmer en assembleur n'est pas très courant mais le faire au début de ses études permet d'apprendre des leçons qui seront utiles dans le futur.

Les étapes de la compilation

Les étapes de la compilation sont les suivantes :



Le langage d'Assemblage (Assembleur)

Contrairement aux langages de haut niveau, une instruction d'assembleur est, en général, équivalente à une instruction en langage machine.

Néanmoins, il est encore une version « lisible pour un humain » du langage machine.

Tant qu'un langage de haut niveau est censé être conçu pour être indépendant de la machine, l'assembleur est fortement dépendant de l'architecture de la machine, comprise comme le jeu d'instructions (CISC ou RISC) supporté par le processeur.

Ainsi, chaque architecture a son propre langage machine et elle peut supporter plusieurs langages d'assemblage.

Dans ce cours, nous allons utiliser le **NASM (Netwide Assembler)**. Il est :

- (Relativement) Indépendant du système d'exploitation ;
- Très utilisé ainsi comme GAS (GNU Assembleur) ;
- Sa syntaxe ressemble (un petit peu) à celle du C ;
- Dispose d'un préprocesseur.

Il existe d'autres compilateurs comme MASM (Microsoft Assembler) qui sont aussi très utilisés dans les systèmes Windows. Néanmoins, la syntaxe de MASM :

- est substantiellement différente de celle de NASM ;
- rend la programmation plus subtile et des erreurs de syntaxe sont plus fréquentes ;
- presque toujours bien comprise par d'autres assembleurs tels que TASM ou CHASM.

Différences entre NASM et GNU assembleurs

NASM utilise la syntaxe Intel pour le code assembleur. Je considère la syntaxe de du GNU Assembleur et AT & T Assembleur plus lourde. Quelques différences par rapport à NASM sont les suivantes :

1. Les registres commencent par le caractère % ;
2. Les littéraux commencent par le caractère \$;
3. La destination de l'opération est le deuxième opérande et pas le premier ;
4. Les instructions peuvent être complétées avec la taille des opérandes ;
5. La syntaxe utilisée pour le calcul des adresses effectives est différente.

Compilation et linkage avec NASM

Si vous travaillez sur votre ordinateur, il faut commencer par [installer nasm](#). Ensuite ...

Pour assembler et générer un exécutable dans 32 bits ELF (Executable and linkable format) il faut :

- L'assemblage du code dans un fichier objet :
nasm -f elf32 -o <nom de fichier>.o <nom de fichier>.asm
- La liaison du code la création d'un exécutable :
ld -m elf_i386 -o <nom de fichier> <filename>.o

Pour assembler et générer un exécutable assembleur dans 64 bits ELF :

- Pour l'assemblage du code dans un fichier objet :
nasm -f elf64 -o <nom de fichier>.o <nom de fichier>.asm
- La liaison du code la création d'un exécutable :
ld -o <nom de fichier> <nom de fichier>.o

Compilation et linkage avec NASM

Quelques considérations techniques importantes :

A partir de la version de GNU Binutils 2.39 Released disponible à partir de l'année 2023, le linker ELF va générer un avertissement si la pile devient exécutable.

C'est une aide pour identifier des programmes qui sont potentiellement vulnérables à des attaques sur des zones de mémoire exécutables.

Pour éviter que l'exécutable généré dispose d'une pile exécutable, il est possible d'utiliser le flag **noexecstack** au moment du linkage comme suit :

```
ld -z noexecstack -o <nom de fichier> <nom de fichier>.o
```

Ou

```
g++ -z noexecstack -o <nom de fichier> <nom de fichier>.o
```


La structure d'un programme NASM

Il y a trois sections :

section.data : elle est utilisée pour déclarer :

- Des constantes ;
- Des données initialisées. Leur valeur peut changer tout au long du programme.

section.bss : elle permet de déclarer des variables comme des espaces en mémoire tels que :

- Des données non initialisées. Leur valeur peut changer tout au long du programme.

section.text : elle contiendra le code du programme.

- Elle doit commencer avec une déclaration

global _start

_start:

Commentaires : Les commentaires commencent avec la point virgule (;) et s'étalent uniquement sur une ligne.

```
section .text
```

```
global _start ; Cette déclaration est nécessaire pour le linkage.
```

```
_start: ; C'est le point d'entrée du programme.
```

```
mov     edx,len ; Longueur du message à afficher.
```

```
mov     ecx,msg ; Message à afficher.
```

```
mov     ebx,1 ; description de la sortie standard (stdout)
```

```
mov     eax,4 ; Code pour la écriture (sys_write)
```

```
int     0x80 ; Appel au noyau (aussi 80h).
```

```
mov     eax,1 ; Code pour la fin du programme
```

```
int     0x80 ; Appel au noyau
```

```
section .data ; Section pour les données.
```

```
msg db "Hello, world!", 0xa ; Chaîne de caractères affichée.
```

```
len equ $ - msg ; Longueur de la chaîne de caractères affichée.
```

Les registres généraux (1/2)

RAX, EAX, AX, AH, AL : Ce sont les registres d'accumulation (accumulator register). Utilisés pour :

- Définir les ports d'entrée/sortie ;
- Le résultat des calculs arithmétiques ;
- Des appels aux interruptions.

EBX, BX, BH, BL : C'est le registre de base (base registre). Utilisé pour :

- Pointeur de base pour les accès en mémoire ;
- Récupérer certaines valeurs de retour.

XMM0 à XMM15 : Ce sont des registres à 128 bits permettant de stocker des nombres à virgule flottante avec la représentation **IEEE 754** à simple ou à double précision.

Les registres généraux (2/2)

ECX,CX,CH,CL : C'est le registre de compteur (counter registre). Utilisé comme :

- Compteur de boucles et pour des shifts ;
- Récupère certaines valeurs des interruptions.

EDX,DX,DH,DL : C'est le registre des données (data registre). Il est utilisé pour :

- Spécifier des ports pour l'accès d'entrée/sortie ;
- Certaines opérations arithmétiques comme la division et la multiplication ;
- Fixer certaines valeurs des appels aux interruptions.

Très important : Ces registres « généraux » ont parfois des utilisations très spécifiques pour certaines opérations arithmétiques et d'entrée/sortie. Ainsi, il faut faire très attention aux conditions dans lesquelles les opérations sont réalisées et quels sont les registres utilisés.

Les registres de segment

Ces registres conservent les adresses de base des segments. Ils sont disponibles dans 16 bits :

- **CS** : Conserve l'adresse du segment du code (segment text) dans lequel s'exécute le programme.
 - **Attention** : changer sa valeur peut changer l'exécution de votre programme.
- **DS** : Conserve l'adresse du segment de données (segment data) utilisé par le programme.
 - **Attention** : changer sa valeur peut changer les données qui sont accessibles par votre programme.
- **ES,FS,GS** : Ce sont des registres de segment additionnels. Ils sont disponibles pour :
 - Accéder à la mémoire vidéo qui ne se trouve pas dans le segment de base du programme.
- **SS** : Conserve l'adresse du segment de la pile(stack segment) utilisée par le programme.
 - Il peut avoir la même valeur que le registre général DS ;
 - **Attention** : Changer sa valeur peut générer des comportements inattendus dans le traitement et l'accès aux données du programme.

Les registres comme des indexes et des pointeurs (1/2)

Ces registres sont généralement utilisés comme :

- Des indices ou des pointeurs vers des adresses ;
- Un décalage par rapport à une adresse ;
- Les registres avec une « E » au début ne peuvent pas être utilisés qu'en **mode protégé**.

Ils sont plusieurs mais chacun a des fonctions spécifiques telles que :

- **ES:EDI EDI DI** : Registre indice de destination. Utilisé pour
 - Manipuler des string (chaînes de caractères), la mémoire, faire des copies des arrays ;
 - Pour le stockage des adresses d'entrée/sortie.
- **DS:ESI EDI SI** : Stockage de l'indice d'origine. Utilisé pour :
 - Faire des copies des strings ou des segments de mémoire.

Les registres comme des indexes et des pointeurs (2/2)

Ces registres sont généralement utilisés comme :

- Des indices ou des pointeurs vers des adresses ;
- Un décalage par rapport à une adresse ;
- Les registres avec une « E » au début ne peuvent pas être utilisés qu'en **mode protégé**.

Ils sont plusieurs mais chacun a des fonctions spécifiques telles que :

- **SS:EBP, BP** : Stockage du pointeur vers la base (adresse la plus élevée) de la pile (Stack).
 - Il maintient l'adresse de la base de la pile.
- **SS:ESP, SP** : Stockage du pointeur vers la partie haut (adresse la moins élevée) de la pile
 - Il maintient l'adresse du haut de la pile.
- **CS:EIP, IP** : Registre utilisé comme indice pour les instructions du programme :
 - Maintien le décalage nécessaire pour l'instruction suivante du programme ;
 - **Attention** : il n'est pas accessible qu'en mode lecture.

Les registres de signalisation - Eflags (1/2)

Ces registres sont utilisés pour stocker l'état du processeur :

- Ils sont modifiés par plusieurs instructions ;
- Ils sont utilisés dans la comparaison des valeurs pour des sauts conditionnels et dans les conditions des boucles de répétition ;
- Chaque bit conserve l'état d'un paramètre spécifique à la dernière instruction exécutée.

Bit	Etiquette	Description
0	CF	Bit de report (carry) arithmétique
2	PF	Bit de parité
4	AF	Bit de report (carry) arithmétique auxiliaire
6	ZF	Bit de zéro
7	SF	Bit de signe

Les registres de signalisation - Eflags (2/2)

Bit	Etiquette	Description
8	TF	Bit de trap
9	IF	Bit de habilitation d'interruption
10	DF	Bit de direction
11	OF	Bit de dépassement (overflow)
12-13	IOPL	Bits gérant les niveaux de privilèges pour l'entrée/sortie
14	NT	Bit de tâche imbriquée
16	RF	Bit de résumé (de programme)
17	VM	Bit de mode virtuel 8086
18	AC	Bit de vérification d'alignement (pour les x486 et plus)
19	VIF	Bit d'interruption virtuelle
20	VIP	Bit d'attente (pending) d'interruption virtuelle

Le langage NASM assembler

L'ensemble de caractères utilisable est :

- Caractères **a..z** ou **A..Z** en incluant les parenthèses () ;
- Nombres de **0..9** ;
- Caractères spéciaux comme ?, _, @, etc.

NASM est sensitive à la case pour les étiquettes et les variables. Ce qui n'est pas le cas de la majorité de langages Assembler.

NASM n'est pas sensitive à la case pour les mots clés instructions (mnemonics), noms de registre et les directives. Par exemple :

MOV EAX, 1 et équivalent à **mov eax, 1**

Les éléments de base : Littéraux

Les littéraux sont des valeurs qui peuvent être connues ou calculées au moment de l'assemblage du code.

Par exemple :

- Des chaînes de caractères codées en dur entre guillemets simples ou double guillemets. :
 - 'Tout s'est bien passé'
 - "L'opération a réussie"
- Des constantes qui peuvent être des entiers, des réels ou des adresses ;
 - 3.1416 (la valeur de PI) ;
 - 0FAAh ;
 - 0x1A01

Les entiers sont des valeurs numériques sans décimaux. Ils peuvent inclure des qualificatifs pour spécifier la base dans laquelle le nombre est exprimé :

- **b** ou **y** pour binaire ;
- **d** pour décimal ;
- **h** pour hexadécimal ;
- **q** pour octal.

Par exemple :

- 12d est un décimal ;
- 15h ou 0Fh est un hexadécimal. **Attention** : les hexadécimales commençant avec A-F, doivent avoir un 0 avant ;
- 1010b est un nombre binaire.

A remarquer : des syntaxes comme 0xc8 sont aussi acceptées.

Les instructions de base

Dans assembleur, les instructions ont la forme suivante :

[étiquette [:]] [mnemonic (ou instruction)] [opérandes] [; Possibles commentaires]

L'étiquette peut aussi s'appeler **nom**. Elles sont utilisées pour :

- Définition des structures de données ;
- Pour identifier une localisation dans le code.

Toutes les parties dans cette structure sont optionnelles et les lignes vides sont valables.

Chaque instruction doit être contenue dans une seule ligne et il ne peut pas avoir plus de 128 caractères.

Très important : les instructions ne peuvent pas avoir deux opérandes de type mémoire. Il faut passer par les registres.

Pour clarté, les étiquettes, apparaissent très suivant dans une ligne séparée :

Somme:

ADD eax, edx ; Ajoute deux valeurs et les laisse dans eax

Les étiquettes permettent d'identifier des variables, des symboles et aussi des mots clés. Elles peuvent contenir :

- Des lettres A..Z et a..z ;
- Des nombres 0..9 ;
- Des caractères spéciaux tels que ?, _, @, \$, etc.

Le premier caractère doit être une lettre, un tiré bas (_) ou un point (.) Le point dans l'étiquette spécifie que l'étiquette est locale, cela veut dire qu'elle peut être redéfinie.

Attention : les étiquettes ne peuvent pas correspondre aux mots réservés du langage.

Les types d'instruction : les directives

Les directives :

- Dirigées à l'assembleur et pas au processeur ;
- Dit à l'assembleur de faire quelque chose ou pour l'informer de quelque chose ;
- Inclure d'autres fichiers ;
- La définition de mémoire pour stocker des données ;
- Inclure des codes sources en se basant sur des conditions ;
- Ne sont pas traduites en code machine.

NASM possède un **préprocesseur**, il y a beaucoup de commandes identiques à celles du préprocesseur C.

Par contre, les directives NASM commencent par % et pas par # comme en C.

Les types d'instruction : les directives

NASM possède un **préprocesseur**, il y a beaucoup de commandes identiques à celles du préprocesseur C.

Par contre, les directives NASM commencent par % et pas par # comme en C.

Des exemples :

limit EQU 100 ; Définition d'un **symbole**. Affecte la valeur de 100 à la symbole limit et ; se comporte comme une constante.

%define limit 100 ; C'est équivalent à un #define en C/C++. Permet de définir des ; macros et peuvent être redéfinies plus tard.

CPU P4 ; Demande d'utiliser le jeu d'instruction du Pentium 4.

Les types d'instruction : la définition des données

Les directives de données ou définitions de données : elles sont utilisées dans les segments de données pour :

- Réserver de la place en mémoire ;
- Allouer uniquement l'espace en mémoire ou de l'allouer en donnant une valeur initiale ;
- Affecter des étiquettes pour faire référence aux emplacements de mémoire dans le code.

Les directives de base sont :

- **RESX** : Pour réserver uniquement l'espace en mémoire avec **X** pour spécifier la taille des objets qui seront stockés ;
- **DX** : Pour réserver avec une valeur initiale avec **X** pour spécifier la taille des objets qui seront stockés.

Les types d'instruction : la définition des données

Les directives de données ou définitions de données :

- **RESX** : Pour réserver uniquement l'espace en mémoire avec **X** pour spécifier la taille des objets qui seront stockés ;
- **DX** : Pour réserver avec une valeur initiale avec **X** pour spécifier la taille des objets qui seront stockés.

La valeur de **X** dépendra de la taille de ce qui veut être stocké. Les valeurs utilisables sont les suivantes :

- **B** : Un octet ;
- **W** : Un mot (deux octets);
- **D** : Un double mot ;
- **Q** : Un quadruple mot (à utiliser uniquement pour les constants à virgule flottante en double précision) ;
- **T** : Dix octets.

Les types d'instruction : la définition des données

Des exemples :

L1 db 0	; Initialise un emplacement L1 d'un octet (byte) avec un zéro.
L2 db 101010b	; Initialise un emplacement L2 d'un octet (byte) avec un nombre binaire.
L3 dw 1002	; Initialise un emplacement L3 d'un mot (16 bits) avec la valeur 1002.
L4 resb 1	; Alloue la mémoire pour un seul emplacement L4 d'un octet (byte).
L5 db "A"	; Initialise un emplacement L5 d'un octet (byte) avec la valeur ASCII A (65).
L6 db 1, 2, 3, 4	; Initialise quatre emplacements L6 d'un octet (byte) avec les valeurs données.

Très important, les définitions de données consécutives sont stockées séquentiellement en mémoire. C'est à dire que L3 vient immédiatement après L2.

Les types d'instruction : la définition des données

La directive **TIMES** permet de répéter un opérande une certaine quantité des fois.

Par exemple :

Nombres times 100 db 1 ; C'est équivalent à db utilisée 100 fois.
Mots resw 100 ; Réserve la place pour 100 mots (16 bits).

Pour déclarer des chaînes de caractère :

Mot0 db "m", "o", "t", 0 ; Permet de définir une chaîne de caractères "mot".
Mot1 db "mot", 0 ; C'est pareil que l'instruction précédente.

Attention : la directive **DQ** permet de définir que des constantes à virgule flottante en double précision.

Les types d'instruction : les instructions

Les instructions :

- Permettent de manipuler des étiquettes, des registres, des variables, des littéraux et d'autres types de données pour réaliser des opérations ;
- Le nombre et type d'opérandes sont spécifiques à chaque instruction ;
- Les opérandes peuvent être des valeurs mais aussi des adresses en mémoire ;
- En général, l'adresse de destination de l'opération se trouve comme premier opérande.

Les affectations avec MOV

L'instruction **MOV** permet de déplacer la valeur d'une variable ou d'une étiquette dans un des registres du processeur ou dans l'espace alloué à une autre étiquette. Elle a la syntaxe suivante :

MOV <destination>, <origine>

Par exemple :

MOV AL, [E1]	; Copie l'octet affecté à E1 dans le registre al (8 bits).
MOV eax, E1	; Copie l'adresse de l'octet en E1 dans eax (32 bits).
L1 DW 112	; Allocation d'un espace en mémoire de 16 bits contenant 112.
MOV AL, [L1]	; Copie le premier octet de L1 dans le registre AL.

Dans la dernière ligne, il faut constater qu'uniquement les 8 bits les moins significatifs L1 seront stockés dans AL.

Important : c'est la responsabilité du programmeur de s'assurer que les données sont toujours affectés aux bons espaces en mémoire, NASM ne garde pas de trace de la taille des données.

Les affectations avec MOV

Le stockage des littéraux peut poser des problèmes comme dans le cas suivant :
Pour l'instruction :

MOV [L1], 1 ; Stocker 1 en L1.

Assembleur va produire une erreur du type « **operation size not specified** ».
Cela est dû au fait que NASM pourrait stocker 1 comme un octet, un mot ou un double mot.
Solution : spécifier quelle est la taille de stockage de cette valeur comme suit :

MOV DWORD [L1], 1 ; Pour stocker 1 comme un double mot.

Les autres spécificateurs sont : BYTE, WORD, QWORD et TWORD (celui pour un espace en mémoire de 10 octets).

Les étiquettes comme des adresses

En principe, les étiquettes sont des adresses où se trouvent stockées les données.

Par contre, pour faire référence à **la valeur stockée** dans l'adresse référencée par l'étiquette **E1**, il faut mettre le nom de l'étiquette entre crochets **[E1]**.

En d'autres termes, l'étiquette est un pointeur vers la donnée et l'opérateur [] agit comme l'opérateur de déréréférencement de ce pointeur.

Par exemple :

E1 db 10	; Affectation de la valeur 10 à l'étiquette E1.
MOV AL, [E1]	; Copie l'octet affecté à E1 dans le registre AL (8 bits).
MOV EAX, E1	; Copie l'adresse de l'octet en E1 dans le registre EAX (32 bits).
MOV [E1], AH	; Copie le contenu d'AH dans l'octet de E1.

Variables et constantes

Une étiquette est considérée une espèce de **variable** si sa valeur peut être changée tout au long du programme. C'est le cas des étiquettes déclarées avec les instructions **DX** (Segment Data) et **RESX** (Segment BSS).

Par exemple :

V1 db 10 ; V1 est une variable d'un octet initialisée avec la valeur 10.
V2 resw 2 ; V2 est l'adresse initiale de deux variables d'un mot sans valeur initiale.

Une étiquette est considérée **une constante** si sa valeur ne peut pas être changée une fois qu'elle a été initialisée.

Par exemple :

Max EQU 100 ; Constante Max contenant la valeur 100. Ce n'est pas stockée en mémoire mais la valeur est remplacée là où l'étiquette Max apparaît dans le programme.

ETI :

MOV eax, 0 ; ETI est l'adresse de l'instruction.
JUMP ETI ; Instruction pour sauter à l'instruction précédente.
Lcounter EQU \$; Le caractère spécial \$ signifie l'adresse de l'instruction courante (location counter).

The location counter

Le location counter est incrémenté pour chaque octet qui est ajouté au fichier objet.

Il est incrémenté pendant le processus d'assemblage du code assembleur. Ce processus envoie du code ou des données au fichier objet.

Par exemple :

ETI : MOV eax, 0	; ETI est l'adresse de l'instruction.
JUMP ETI	; Instruction pour sauter à l'instruction précédente.
Lcounter EQU \$	; Le caractère spécial \$ signifie l'adresse de l'instruction courante (location counter).

The location counter

Le caractère spéciaux \$ pointe vers l'adresse en mémoire actuelle du programme.

Exemple d'utilisation :

```
str1 db 'Voici une chaîne de caractères' ; La variable message contient 30 octets.  
slen EQU $ - str1 ; La constante slen va contenir 30, qui est  
; la longueur de str1 parce qu'elle pointe vers  
; l'adresse du premier octet de la chaîne de  
; caractères.
```

Important : Le registre IP contiendra l'adresse de la prochaine instruction à exécuter dans le programme

L'entrée/sortie avec interruptions

En principe, il n'est possible d'afficher que des informations représentables comme des codes ascii.

Ainsi, les nombres entiers dans la sortie seront toujours interprétés comme des codes Ascii.

C'est-à-dire, des codes ASCII à 1 octet, même si à la base, l'espace en mémoire doit représenter un entier ou un nombre à virgule flottant.

Conclusion : Les nombres ne peuvent pas être affichés que chiffre par chiffre.

Solution : utiliser des fonctions C/C++ pour afficher les différents types d'information.

L'entrée/sortie avec interruptions

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Les modes d'adressage

Modes d'adressage : Adressage des registres

Il existe plusieurs modes d'adressage en Assembleur :

Adressage des registres : dans ce cas, au moins un registre est utilisé comme opérande ;

- **MOV [Counter], CX** : le deuxième opérande est un registre ;
- **MOV EAX, EBX** : Les deux opérandes sont des registres.

Modes d'adressage : adressage direct

Adressage direct : Le deuxième opérande est un constant ou une valeur. Attention, la taille du premier opérande définit la taille de la donnée stockée :

- **VALEUR_OCTET DB 200** : Une valeur est stockée dans un octet ;
- **ADD [VALEUR_OCTET], 50** : Une valeur constante est ajoutée à la valeur dans la constante ;
- **MOV AL, 45d** : Une valeur constante est copiée dans un registre ;

Modes d'adressage : adressage direct dans la mémoire

Adressage direct dans la mémoire : il a besoin d'un accès directe au segment de définition de données.

- C'est une manière plus lente d'accéder aux données ;
- L'adresse dans laquelle on va stocker ou lire la donnée est écrite directement dans l'instruction;
- En général, une étiquette est utilisée à la place d'une adresse directe en mémoire;
- Les étiquettes d'adresses utilisées sont déclarées et définies à l'intérieur du segment de données du programme.

Au moins un des opérandes pour ces instructions doit faire référence à un registre tel que :

- **ADD [VALUE_OCTET], DL**
- **MOV BX, [VALUE_MOT]**

Modes d'adressage : utilisation des décalages

Adressage de la mémoire par décalage : Il est possible d'utiliser des opérateurs arithmétiques pour accéder à des adresses dans le segment de données. Pour les tableaux suivants :

TABLEAU_OCTETS DB 15, 16, 23, 33 ; Un tableau d'octets.

TABLEAU_MOTS DW 124, 231, 290, 121 ; Un tableau des mots (16 bits).

Il est possible d'accéder aux informations avec les instruction suivantes :

MOV CL, [TABLEAU_OCTETS + 2] ; Obtient aussi le troisième octet dans
TABLEAU_OCTETS

MOV CX, [TABLEAU_MOTS + 6] ; Obtient aussi le quatrième mot dans TABLEAU_MOTS

Modes d'adressage : adressage indirect

Adressage indirect de la mémoire : Ce mode utilise les :

- Registres de base **RBX, EBX, RBP EBP, BX** et **BP** ;
- Et les registres d'indice **DI** et **SI**.

Pour coder les adresses en mémoire entre crochets. Il est généralement utilisé pour des variables qui contiennent plusieurs valeurs tels les **Arrays**. Par exemple :

TABLEAU TIMES 10 DW 0 ; Allocation de 10 mots en mémoire avec la valeur 0.

MOV EBX, TABLEAU ; Les crochets permettent d'obtenir l'adresse effective du tableau TABLEAU.

MOV [EBX], 50d ; TABLEAU[0] = 50

ADD EBX, 2 ; Nous augmentons l'adresse en EBX de deux octets. Attention, ces sont des octets et pas de mots.

MOV [EBX], 15 ; TABLEAU[1] = 15

Modes d'adressage : Position Independent Executable

Position Independent Executable (PIE) : C'est un exécutable qui peut être chargé de manière dynamique. Ainsi, les adresses en mémoire qu'il utilise ne sont pas déterminées de manière absolue dans la mémoire mais de manière relative au segment courant d'exécution du programme.

Le flag **-no-pie** à l'édition des liens permet de dire que l'exécutable à générer n'est pas indépendant de la position en mémoire.

L'instruction **rel** (relatif), avant une adresse, permet de générer un adressage relatif par rapport à la valeur dans le registre d'instructions courant (**RIP**).

L'instruction **LEA (Load Effectif Address)** met l'adresse effective du deuxième opérande et le met dans le premier. Par exemple: **LEA RSI, [rel ADRESSE]**

Les instructions arithmétiques

Lecture des arguments de l'invité de commandes

Il est possible de récupérer les arguments fournis par l'utilisateur via l'invité de commandes.

Une main n'est qu'une fonction, ainsi pour

```
int main(int argc, char** argv)
```

Les arguments :

- **argc** sera en RDI ;
- **argv** (un pointeur) sera en RSI.

```
global main
```

```
extern puts
```

```
section .text
```

```
main:
```

```
push    rdi                ; Stocke des registres
```

```
push    rsi                ; utilisés par puts.
```

```
sub     rsp, 8             ; Alignement du stack pointeur.
```

```
mov     rdi, [rsi]         ; the argument string to display
```

```
call    puts              ; print it
```

```
add     rsp, 8             ; restore %rsp to pre-aligned value
```

```
pop     rsi               ; Récupération des registres
```

```
pop     rdi               ; utilisés par puts.
```

```
add     rsi, 8             ; Passage à la position de l'argument suivant.
```

```
dec     rdi               ; Réduit la valeur de RDI (nombre  
d'arguments).
```

```
jnz     main              ; Si RDI n'est pas zéro, répète la boucle.
```

Les instructions arithmétiques

Elles permettent de réaliser des opérations mathématiques. Leur principales caractéristiques sont :

- Elles ont une opérande destination et une opérande source ;
- C'est l'opérande destination qui détermine la taille de ce qui sera finalement stocké ;
- Elles peuvent être réalisées avec des registres à 8, 16, 32 et 64 bits ;
- Il n'est pas possible de réaliser des opérations entre deux opérandes résidant en mémoire, il faut toujours passer par des registres ;
- En dépendant de l'opération, elles peuvent affecter ou réinitialiser les valeurs des indicateurs de débordement (overflow) ou de carry.

INC et DEC

INC : Permet d'incrémenter l'opérande par une valeur de 1.

MOV EAX, 0 ; Initialisation du registre.

INC EAX ; Augmentation de sa valeur en 1.

INC [variable] ; Incrémente la valeur de la valeur contenue en variable.

DEC : Décrémente la valeur de l'opérande par une valeur de 1.

MOV EAX, 10 ; Initialisation du registre.

DEC EAX ; EAX vaut 9 maintenant.

DEC [variable] ; Décrémente la valeur de la valeur contenue en variable.

ADD et SUB

Elles permettent de réaliser des additions ou des soustractions. Leur forme générale est :

ADD <destination>, <Origine>

Elles peuvent être utilisées :

- Registre à registre
- Mémoire à registre ;
- Registre à mémoire ;
- Données constantes à des registres ;
- Données constantes à des espaces en mémoire.

```
segment .data :
```

```
    constant db 15
```

```
segment .bss :
```

```
    num resb 4 ; Variable sur quatre octets.
```

```
section .text :
```

```
    global _start
```

```
_start :
```

```
    MOV EAX, 5
```

```
    MOV EBX, 10
```

```
    ADD EAX, EBX ; EAX contiendra la valeur 15.
```

```
    ADD [num], EAX ; num contiendra 15 dans 4 octets.
```

```
    SUB [num], EBX ; num contiendra 5 dans 4 octets.
```

```
    ADD [num], constant ; num = 20.
```


MUL et IMUL

MUL : permet de multiplier des nombres sans signe.

IMUL : Multiplie des nombres avec signe.

Sa syntaxe de base est :

MUL <Multiplicateur>

Parce que le **multiplicande** et le **résultat** sera toujours dans le registre **EAX**.

Cependant, il est possible d'utiliser AL (8 bits), AX (16 bits), EAX (32 bits) et RAX (64 bits) en dépendant de la taille des opérandes.

Le multiplicateur peut être en mémoire ou dans un autre registre.

```
segment .data :  
    constant db 10  
  
segment .bss :  
    res resb 4 ; Variable sur quatre octets.  
  
section .text :  
    global _start  
  
_start :  
  
    MOV AL, '3'  
    SUB AL, '0' ; Transformation dans un entier.  
    MOV bl, '2'  
    SUB bl, '0' ; Transformation dans un entier  
    MUL BL  
    ADD AL, '0' ; Transformation dans un entier  
    MOV [res], AL  
    MOV ECX,msg  
    MOV EDX, len  
    MOV EBX, 1 ; Descripteur de la sortie standard (stdout)  
    MOV EAX, 4 ; Code de l'appel système (sys_write)  
    INT      0x80 ; Appel au kernel
```

MUL et IMUL

En dépendant de la taille des opérateurs :

AL x <multiplicateur à 8 bits> = AH AL

AX x <multiplicateur à 16 bits> = DX AX

EAX x <multiplicateur à 32 bits> = EDX EAX

Attention : l'ordre des registres est important, les bits les moins significatifs seront dans les registres à droite, AL, AX et EAX respectivement

```
segment .data :
```

```
    constant db 10
```

```
segment .bss :
```

```
    res resb 4 ; Variable sur quatre octets.
```

```
section .text :
```

```
    global _start
```

```
_start :
```

```
    MOV AL, '3'
```

```
    SUB AL, '0' ; Transformation dans un entier.
```

```
    MOV bl, '2'
```

```
    SUB bl, '0' ; Transformation dans un entier
```

```
    MUL BL
```

```
    ADD AL, '0' ; Transformation dans un entier
```

```
    MOV [res], AL
```

```
    MOV ECX,msg
```

```
    MOV EDX, len
```

```
    MOV EBX, 1 ;Descripteur de la sortie standard (stdout)
```

```
    MOV EAX, 4 ;Code de l'appel système (sys_write)
```

```
    INT      0x80 ;Appel au kernel
```

DIV et IDIV : division entière sans signe

DIV : permet de diviser des nombres sans signe.

IDIV : Divise des nombres avec signe.

Attention : à 64 bits, la taille par défaut de l'opération est de 32 bits.

Sa syntaxe de base est :

DIV <diviseur>

Parce que le **dividende** sera toujours dans un registre dépendant de sa taille : AL (8 bits), AX (16 bits), EAX (32 bits) et RAX (64 bits).

Le **diviseur** peut être en mémoire ou dans un autre registre.

DIV et IDIV : division entière sans signe

En dépendant de la taille du dividende :

8 ou 16 bits : Dividende en AX et diviseur à 8 bits => Résultat en AL et le reste en AH

16 à 32 bits : Dividende en DX:AX. Les bits les plus significatifs sont en DX et les moins significatifs en AX. Le diviseur à 16 bits => Résultat de 16 bits en AX et le reste à 16 bits en DX

32 à 64 bits : Dividende à 64 bits en EDX:EAX (64 bits). Les bits les plus significatifs sont en EDX et les moins significatifs en EAX. Le diviseur à 32 bits => Résultat de 32 bits en EAX et le reste à 32 bits en EDX

64 à 128 bits : Dividende à 64 bits en RDX:RAX (128 bits). Les bits les plus significatifs sont en RDX et les moins significatifs en RAX. Le diviseur à 64 bits => Résultat de 64 bits en RAX et le reste à 32 bits en RDX

DIV et IDIV : division entière sans signe

section .data

message db "Le résultat est :", 0xA,0xD

longueur equ \$- message

segment .bss

res resb 1

section .text

global _start ; Déclaration pour le début du programme.

_start: ; Début du programme.

mov ax, '8'

sub ax, '0' ; Convertit l'ASCII en entier entre 0 et 9.

mov bl, '2'

sub bl, '0' ; Convertit l'ASCII en entier entre 0 et 9.

div bl ; Dividende à 8 bits.

add ax, '0'

mov [res], ax ; Le résultat est en AL. Le reste en AH.

mov ecx, message

mov edx, longueur

mov ebx, 1 ; Descripteur de la sortie standard (stdout)

mov eax, 4 ; Appel pour la sortie (sys_write)

int 0x80 ; Appel au kernel

mov ecx, res

mov edx, 1

mov ebx, 1 ; Descripteur de la sortie (stdout)

mov eax, 4 ; Appel pour la sortie (sys_write)

int 0x80 ; Appel au kernel

mov eax, 1 ; Code pour la fin du programme (sys_exit)

int 0x80 ; Appel au kernel

Les instructions logiques et les structures conditionnelles

Les opérateurs logiques de base

Les instructions de logiques de base sont les suivantes :

- **AND** : AND <opérandeA>, <opérandeB>
- **OR** : OR <opérandeA>, <opérandeB>
- **XOR** : XOR <opérandeA>, <opérandeB>
- **NOT** : NOT <opérande>
- **TEST** : TEST <opérandeA>, <opérandeB>

Les premiers et deuxièmes opérandes peuvent être des registres ou des positions en mémoire. Le deuxième opérande peut être aussi une constante.

Ces opérations se font bit par bit comme cela serait le cas d'un circuit logique.

Important : Le résultat de l'opération est laissé dans le premier opérande sauf dans le cas de l'opération TEST.

Exemple des opérateurs logiques

```
section .data
    pair_msg db 'Nombre pair' ; Message impair.
    len1 equ $ - pair_msg
    impair_msg db 'Nombre impair!' ; Message
    impair.
    len2 equ $ - impair_msg
section .text
    global _start
_start:                ;Point d'entrée du programme.
    mov ax, 8h          ;Met 8 dans AX.
    and ax, 1           ; Opération AND.
    jz Pair             ; Va a l'étiquette Pair si la
                        ; valeur de l'opération est zéro.
```

```
    mov eax, 4           ; sys_write
    mov ebx, 1           ; stdout
    mov ecx, odd_msg     ;Message à écrire.
    mov edx, len2        ;Longueur du message.
    int 0x80             ;Appel au kernel
    jmp Finir
```

Pair:

```
    mov ah, 09h
    mov eax, 4           ;sys_write
    mov ebx, 1           ;stdout
    mov ecx, even_msg    ;Message à écrire.
    mov edx, len1        ;Longueur du message.
    int 0x80             ;Appel au kernel
```


Les instructions conditionnelles se divisent dans deux types :

- **Les sauts inconditionnels** : C'est implanté par l'instruction JMP. Un saut fait un transfert de contrôle vers une instruction qui ne suit pas celle du JMP. Cela peut être :
 - Une instruction future pour changer le flux du programme ;
 - Une instruction passée pour répéter l'exécution d'un ensemble d'instructions.
- **Les sauts conditionnels** : qui sont réalisés uniquement si une condition est remplie.
 - Ces instructions changent le flux séquentiel du programme en transférant leur contrôle ;
 - Elles changent la valeur de déplacement dans le registre IP.

L'instruction CMP

C'est structure compare deux opérandes :

- Elle fait la soustraction de deux opérandes pour déterminer si les deux sont égaux. A cause de cela, elle compare des valeurs numériques ;
- Elle ne change aucun des deux opérandes ;
- Les deux opérandes peuvent être des registres ou des positions en mémoire. Dans le cas du deuxième opérande, il peut être aussi une constante immédiate ;
- Elle est utilisée en combinaison avec les sauts conditionnels tels que **JE** (Jump if equal) afin de prendre des décisions.

Les structures conditionnelles

Elles sont implantées à l'aide de deux instructions :

- Un CMP qui fait la comparaison entre deux opérandes et ;
- Une instruction JUMP conditionnelle qui dépendra de ce que nous voulons faire avec l'information fournie par le CMP.

Par exemple :

CMP DX, 0 ; Compare le registre DX avec la valeur zéro.

JE L2 ; JE (Jump if equal) Si les deux valeurs sont égaux, le
programme transmettra le contrôle au code à l'étiquette L2.

L2:

...

Les différents sauts conditionnels

Pour les valeurs numériques signées, les JUMPs suivants sont fournis :

Instruction	Description	Flags testés
JE/JZ	Sauter si égale ou zéro	ZF
JNE/JNZ	Sauter si non égale grand / Sauter si non zéro	ZF
JG/JNLE	Sauter si plus grand / Sauter si non moins grand	OF, SF et ZF
JGE/JNL	Sauter si plus grand ou égale / Sauter si non moins	OF et ZF
JL/JNGE	Sauter si moins grand / Sauter si non plus grand ou égale	OF et ZF
JLE/JNG	Sauter si moins grand ou égale/ Sauter si non plus grand	OF, SF et ZF

Les différents sauts conditionnels

Pour les valeurs numériques non signées, les JUMPs suivants sont fournis :

- Il s'agit de la même nomenclature mais avec Below au lieu de less et Above au lieu de Greater.

Instruction	Description	Flags testés
JE/JZ	Sauter si non égale ou non zéro	ZF
JNE/JNZ	Sauter si non égale grand / Sauter si non zéro	ZF
JA/JNBE	Sauter si plus grand / Sauter si non moins grand	CF et ZF
JAE/JNB	Sauter si plus grand ou égale / Sauter si non moins	CF
JB/JNAE	Sauter si moins grand / Sauter si non plus grand ou égale	CF
JBE/JNA	Sauter si moins grand ou égale / Sauter si non plus grand	AF et CF

Les différents sauts conditionnels

Il existe des JUMPs permettant de tester directement les flags :

Instruction	Description	Flags testés
JXCZ	Sauter si CX est zéro	Aucun
JC	Sauter s'il y a du carry	CF
JNC	Sauter s'il n'y a pas du carry	CF
JO	Sauter si débordement	OF
JNO	Sauter s'il n'y a pas de débordement	OF
JP/JPE	Sauter si pair	PF
JNP/JPO	Sauter non pair	PF
JS	Sauter si la parité est négative	SF
JNS	Sauter si la parité est positif	SF

Exemple des sauts conditionnels

_start: ;Début du programme

```
mov ecx, [num1]
cmp ecx, [num2]
jg Verifier_3_Nombre
mov ecx, [num2]
```

Verifier_3_Nombre:

```
cmp ecx, [num3]
jg _exit
mov ecx, [num3]
```

_exit:

```
mov [largest], ecx
mov ecx, msg
mov edx, len
mov ebx, 1 ;stdout
mov eax, 4 ;sys_write
int 0x80 ;Appel au kernel
```

```
mov ecx, largest
mov edx, 2
mov ebx, 1 ;stdout
mov eax, 4 ;sys_write
int 0x80 ;Appel au kernel
```

Fin :

```
mov eax, 1
int 80h
```

section .data

```
msg db "Le nombre le plus grand est : ", 0xA, 0xD
len equ $- msg
num1 dd '47'
num2 dd '22'
num3 dd '31'
```

segment .bss

```
largest resb 2
```